In traditional robotics- each robot has its own complex UI /system design which the user is expected to learn in order to interact with the robot.

This is a steep learning curve-People belonging to different backgrounds may not be able to understand the robots technicality. This knocks out the majority of potential end-users who don't want to go through the steep learning curve in order to interact/use the robot.

Here comes Natural Language Processing.

NLM-natural language models help in understanding the intent behind the user's spoken language or a given text.This doesn't stop here the understood intent is then used to perform a robotic action by the given robot.

NLM acts as bridging the gap between translating the human words intent into actionable robot actions.

The whole idea here is the user is totally unaware of the working/technical details about the robot.

The user only specifies what he/she wants -be it asking for some action-which could be to wave hi/place the package from point A to point B or should about anything.

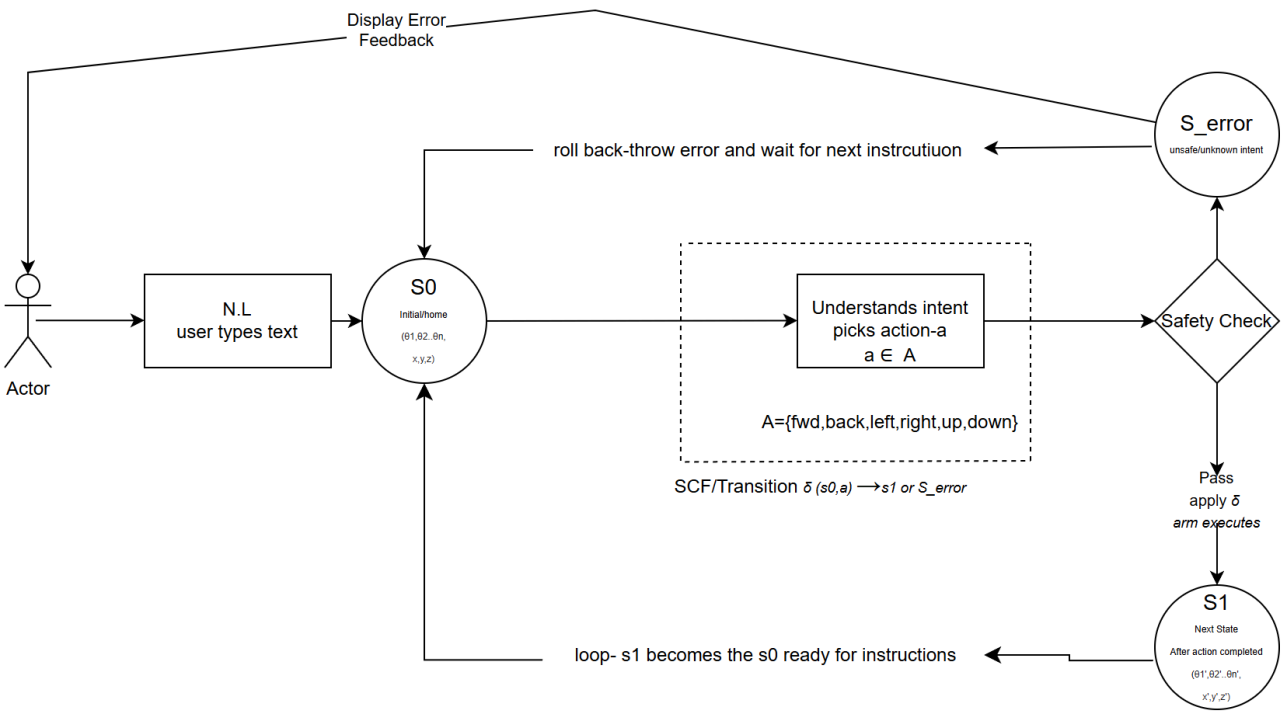
The NLM meanwhile understands the user intent and also has an understanding of the robots physical structure widely known as [URDF]- Unified Robotic Description Format. This essentially contains the understanding of all the Links and joints of the Robot. Lets think of it as a blueprint of the entire Robot which tells us about how much a joint can move. So essentially with the URDF we will know the entire anatomy of the given Robot.

The plan of implementation is simple: we start V0 -- Functional Foundation where the idea is simply to make the RoArm move in the six directions up,down,right and left.

While the six basic commands may sound simple their execution logic is critical.

We use a transition function or also called -state change function **SCF/Transition** $\delta(s0,a) \rightarrow s1 \text{ or } S_error$
Here δ is the symbol for the SCF/transition function. That takes the RoArm from one state to another.

The idea here is as the system evolves rapidly over time - we should have a solid robust architecture that will be able to evolve accordingly where in higher level actions can be boiled down to core V0 actions.For example a high level command like point in the north direction can be broken down into smaller combination of repeated move left and right derived from the core V0 architecture.



State transition diagram

1. What V0 Does

V0 is the Functional Foundation — the simplest thing that actually works end-to-end. You type a natural-language instruction, the arm moves. That's it. No LLM, no computer vision, no grasp planning. Just regex → safety check → HTTP command over WiFi. The point was to prove the architecture first, then replace each box with something smarter later.

The whole thing is built around the state-transition diagram. Every file maps to one box in that diagram. Here's the pipeline as it runs for a single command:

Step	Module	What it does	Diagram box
1	intent_classifier.py	Regex match user text → action name	a belongs to A
2	transition.py	Apply Cartesian delta to get **proposed S'**	δ function (green)
3	safety_checker.py	**Check action is known + S' within bounds**	Diamond
4	arm_executor.py	HTTP GET to arm with JSON command	ARM EXECUTES
5	transition.py	Mark S1, log cycle, reset to S0	Loop arrow

2. config.py — The Single Source of Truth

Everything that would otherwise be a magic number lives here. IP address, JSON command codes, step sizes, safety bounds. This was the first file I wrote — if you change the arm's IP or want to tune the step size, this is the only file you touch.

> **DESIGN DECISION**
> Why put STEP_SIZE_MM = 30.0 here and not in action_deltas directly? Because I wanted to be able to change the step size for all 6 directions at once without hunting through a dict. 30mm came from trial and error — 20mm felt too slow, 50mm was too jerky on the wrist joint.

config.py

```python
# config.py — single source of truth for all magic numbers

ARM_IP = "192.168.4.1"   ← default AP mode IP, change to STA if needed
ARM_PORT = 80
ARM_ENDPOINT = f"http://{ARM_IP}:{ARM_PORT}/js?json="

# RoArm-M2-S JSON command codes
CMD_MOVE_INIT = 100   ← T:100 — go home, all joints reset
CMD_XYZT_GOAL = 104   ← T:104 — Cartesian goal, BLOCKING
CMD_XYZT_DIRECT = 1041   ← T:1041 — non-blocking version

MOVE_SPEED = 80   ← % speed, 0-100; 80 felt right after testing
STEP_SIZE_MM = 30.0   ← 30mm per command step, tuned by hand

# Cartesian action deltas — what changes per command
ACTION_DELTAS = {
    "forward": {"dx": STEP_SIZE_MM, "dy": 0.0, "dz": 0.0},
    "back":    {"dx": -STEP_SIZE_MM, "dy": 0.0, "dz": 0.0},
    "left":    {"dx": 0.0, "dy":  STEP_SIZE_MM, "dz": 0.0},
    "right":   {"dx": 0.0, "dy": -STEP_SIZE_MM, "dz": 0.0},
    "up":      {"dx": 0.0, "dy": 0.0, "dz":  STEP_SIZE_MM},
    "down":    {"dx": 0.0, "dy": 0.0, "dz": -STEP_SIZE_MM},
}

# Safety envelope — hard limits in mm
SAFETY_BOUNDS = {
    "x_min": 50.0,  "x_max": 300.0,
    "y_min": -200.0, "y_max": 200.0,
    "z_min": 50.0,  "z_max": 350.0
}
```

> **NOTE**
> The safety bounds (x: 50–300, y: ±200, z: 50–350) are conservative. They're roughly the arm's reachable envelope minus a 20mm buffer on each side. The ESP32 firmware has its own soft limits too, but we want to catch violations before even sending the command.

3. state.py — The Three Circles

This file is a direct translation of the state-transition diagram into Python. The three StateType enum values map exactly to the three circles: **S0 (rest)**, **S1 (after action)**, **S_error (failed).** ArmState tracks the end-effector position in Cartesian space (x, y, z, t). We don't track joint angles because the RoArm-M2-S ESP32 handles inverse kinematics internally — we just tell it where we want the hand to go.

state.py

```python
class StateType(Enum):   ← 3 circles in the diagram
    S0      = "S0"   ← rest / initial
    S1      = "S1"   ← after action applied
    S_ERROR = "S_error"   ← unsafe / failed

@dataclass
class ArmState:
    x: float   ← mm, forward/back axis
    y: float   ← mm, left/right axis
    z: float   ← mm, up/down axis
    t: float   ← wrist angle in radians/ clamp end-effector open/closing.
    state_type: StateType = StateType.S0

    def copy(self) -> ArmState:
        return copy.deepcopy(self)   ← always copy before computing S'

    def mark_error(self, msg: str):   ← transition to S_error
        self.state_type = StateType.S_ERROR

    def mark_s1(self):   ← action succeeded
        self.state_type = StateType.S1

    def reset_to_s0(self):   ← loop arrow S1→S0
        self.state_type = StateType.S0
```

HOW IT WORKS

The copy() method is the key design decision here. Every time we want to compute a proposed next state S', we call current_state.copy() first. That means if the **safety check fails, we haven't mutated the actual current state.** The arm stays where it is, and we reset cleanly. Forgetting this and mutating S directly was actually the first bug I hit.

4. intent_classifier.py — The AGENT Box

This is the AGENT in the diagram — the layer that figures out what the user wants. V0 uses regex instead of an LLM. The patterns are compiled once at module import time, so the classification itself is essentially zero-latency. The 'first match wins' approach means order matters in _PATTERNS — but for 6 mutually exclusive directions, there's no ambiguity.

intent_classifier.py

```python
# Pre-compiled regex patterns — no LLM needed for V0
_PATTERNS: dict[str, list[str]] = {
    "forward": [r'\bforward\b', r'\bahead\b', r'\badvance\b'],
    "back":    [r'\bback\b', r'\breverse\b', r'\bretreat\b'],
    "left":    [r'\bleft\b', r'\bport\b'],
    "right":   [r'\bright\b', r'\bstarboard\b'],
    "up":      [r'\bup\b', r'\braise\b', r'\blift\b'],
    "down":    [r'\bdown\b', r'\blower\b', r'\bdrop\b'],
}

# compile once at import time — fast on every request
_COMPILED = {
    action: [re.compile(p, re.IGNORECASE) for p in patterns]
    for action, patterns in _PATTERNS.items()
}

def classify_intent(text: str) -> Optional[str]:   ← first match wins
    for action, patterns in _COMPILED.items():
        for pat in patterns:
            if pat.search(text): return action
    return None   ← no match → S_error path triggered upstream
```

5. safety_checker.py — The Diamond

The safety check is the diamond in the state-transition diagram. Two gates in sequence: first check the action is one of the 6 known commands, then check that the proposed S' is inside the workspace envelope. If either fails, the arm doesn't move. Full stop.

safety_checker.py

```python
def check_action_known(action) -> SafetyResult:   ← Gate 1
    if action is None:   ← classifier returned nothing
        return SafetyResult(passed=False, reason="Unknown intent")
    if action not in VALID_ACTIONS:   ← should never happen but good to guard
        return SafetyResult(passed=False)
    return SafetyResult(passed=True)

def check_within_envelope(proposed: ArmState) -> SafetyResult:   ← Gate 2
    b = SAFETY_BOUNDS
    if proposed.x < b["x_min"] or proposed.x > b["x_max"]:   ← x-axis check
        return SafetyResult(passed=False, reason=f"x={proposed.x:.1f}mm out of range")
    # y and z follow the same pattern…
    return SafetyResult(passed=True)

def run_safety_check(action, proposed_state):   ← entry point called by transition.py
    result = check_action_known(action)
    if not result.passed: return result   ← fail fast: no point checking bounds
    result = check_within_envelope(proposed_state)
    if not result.passed: return result
    return SafetyResult(passed=True)
```

DESIGN DECISION

The fail-fast pattern here is intentional. Gate 1 (action known?) runs before Gate 2 (within envelope?) **because checking coordinates for an unknown action is pointless.** Each gate returns early on failure, which keeps the code readable and cheap to execute.

6. arm_executor.py — ARM EXECUTES

This file handles all communication with the physical arm. The RoArm-M2-S runs a simple HTTP server on the ESP32 that accepts JSON commands via GET requests. We serialize our command dict, append it to the endpoint URL, and fire off the request. T:104 (CMD_XYZT_GOAL) is the blocking version — the HTTP response only comes back once the arm has finished moving, which is what we want for V0.

arm_executor.py

```python
def _send_json(cmd: dict) -> ExecutionResult:   ← core comms function
    json_str = json.dumps(cmd, separators=(",", ":")))   ← no spaces in URL!
    url = ARM_ENDPOINT + json_str   ← e.g. http://192.168.4.1/js?json={"T":104,...}

    t0 = time.perf_counter()
    try:   ← measure round-trip latency
        resp = requests.get(url, timeout=ARM_TIMEOUT_SEC)
        latency = (time.perf_counter() - t0) * 1000.0
        if resp.status_code == 200:   ← arm confirms receipt
            return ExecutionResult(success=True, latency_ms=round(latency, 2))

    except requests.exceptions.ConnectionError:   ← WiFi dropped / wrong IP
        return ExecutionResult(success=False, error="Connection failed — is the arm on?")
    except requests.exceptions.Timeout:   ← arm still moving? increase ARM_TIMEOUT_SEC
        return ExecutionResult(success=False, error="Timed out")
```

NOTE

The json.dumps(cmd, separators=(',',':')) trick removes spaces from the JSON string. Without this, a command like {"T": 104} becomes {"T":%20104} after URL encoding, which the ESP32 firmware doesn't handle cleanly. Compact separators avoid that entirely.

7. transition.py — The Full Pipeline

This is the most important file. The delta() function IS the state-transition diagram — it's literally the implementation of δ(S, a) → S'. It calls each of the other modules in order and handles every outcome: safety fail, execution fail, and success. The whole pipeline runs synchronously, which keeps it simple and traceable.

transition.py

```python
def delta(current_state: ArmState, user_text: str) -> TransitionResult:   ← δ(S, a) → S'
    state_before_str = str(current_state)

    # ── Step 1: AGENT ──────────────────────────────────────────
    action = classify_intent(user_text)   ← regex → action name or None


    # ── Step 2: compute proposed S' ────────────────────────────
    proposed = current_state.copy()   ← CRITICAL: don't mutate S, work on a copy
    if action and action in ACTION_DELTAS:   ← apply the Cartesian delta
        d = ACTION_DELTAS[action]
        proposed.x += d["dx"]

    # ── Step 3: SAFETY CHECK (diamond) ─────────────────────────
    safety = run_safety_check(action, proposed)
    if not safety.passed:   ← fail: route to S_error, arm does NOT move
        error_state = current_state.copy()
        error_state.mark_error(safety.reason)
        logger.log_cycle(result)
        error_state.reset_to_s0()   ← S_error → back to S0 (red dashed arrow)
        return result

    # ── Step 4: EXECUTE ────────────────────────────────────────
    exec_result = move_to_position(proposed)   ← sends HTTP GET to arm
    if not exec_result.success:   ← SDK/wifi failure → also S_error
        return result

    # ── Step 5: SUCCESS ────────────────────────────────────────
    proposed.mark_s1()   ← S0 → S1 (green arrow in diagram)
    logger.log_cycle(result)
    proposed.reset_to_s0()   ← loop arrow: S1 becomes new S0
    return result
```

HOW IT WORKS

Notice that both failure paths (safety fail and execution fail) call reset_to_s0() before returning. This is the red dashed recovery arrow in the diagram — S_error collapses back to S0 so the system is always in a consistent state for the next command. There's no 'stuck in error' mode.

8. pipeline_logger.py — The Metrics Layer

The logger is a singleton that's instantiated once at module level in transition.py. Every call to delta() ends with logger.log_cycle(result), which updates the running counters. The summary() method is what the /stats endpoint exposes to the browser UI.

pipeline_logger.py

```python
class PipelineLogger:   ← singleton, module-level in transition.py
    def __init__(self):   ← counters zeroed on start
        self._total = 0
        self._success = 0
        self._latencies: list[float] = []
        self._failures: Counter[str] = Counter()

    def log_cycle(self, result: TransitionResult) -> None:   ← called after every command
        self._total += 1
        if result.safety_passed and result.execution and result.execution.success:   ← only count as success if arm moved
            self._success += 1
            self._latencies.append(result.execution.latency_ms)
        else:   ← bucket failure reasons for diagnostics
            reason = result.safety_reason or (result.execution.error if result.execution else "unknown")
            self._failures[reason] += 1

    def summary(self) -> dict:   ← exposed via /stats endpoint
        return {
            "total_commands": self._total,
            "accuracy_pct": round(self._success / max(self._total, 1) * 100, 1),
            "avg_latency_ms": round(sum(self._latencies) / max(len(self._latencies), 1), 1)
        }
```

NOTE

Using collections.Counter for _failures is convenient — it auto-initialises to 0 for new keys, and its most_common() method makes it easy to report the top failure reasons. In practice, the most common failure mode is 'Unknown intent' (user typed something the classifier doesn't recognise), which is the first thing to improve in V1.

9. app.py — Flask Entry Point

The Flask app is deliberately thin. It holds the global current_state (the live S0) and exposes four endpoints. The real work happens in transition.py — app.py just routes HTTP requests to the right function and formats the response as JSON for the browser UI.

app.py

```
app = Flask(__name__)
current_state = make_home_state()   ← global S0, persists between requests

@app.route("/command", methods=["POST"])   ← main pipeline endpoint
def command():   ← user types, this fires
    global current_state
    user_text = request.get_json()["text"]
    result = delta(current_state, user_text)   ← full 5-step pipeline happens here
    current_state = result.new_state   ← promote S' to new S0 for next command
    return jsonify({"action": result.classified_action, "safety":
result.safety_passed, ...

@app.route("/home", methods=["POST"])   ← returns arm to home coords
def home():  ...   ← calls go_home() in transition.py

@app.route("/ping")   ← quick connectivity check before using UI
def ping():  ...   ← returns {connected: true/false}

@app.route("/stats")   ← returns pipeline summary JSON
def stats():  ...   ← accuracy%, avg latency, failure breakdown
```

HEADS UP

current_state is a module-level global, which works fine for single-user V0 but will cause race conditions if multiple browser tabs send commands simultaneously. The V1 fix is a threading.Lock() around every read-modify-write of current_state. For V0 it's fine — we're controlling one physical arm and one person is driving.

10. Trace: "lift it up a bit"

Let's walk through a single command end-to-end. User types: "lift it up a bit". Here's what happens at each step:

Step	What runs	Input	Output
1 — AGENT	classify_intent()	"lift it up a bit"	"up" (matched \blift\b)
—	get_confidence()	"lift it up a bit", "up"	0.6 (synonym match)
2 — Δ	current_state.copy()	S0: x=235, y=0, z=234	proposed: x=235, y=0, z=234
—	apply ACTION_DELTAS	ACTION_DELTAS["up"].dz = +30	proposed.z = 264
3 — SAFETY	check_action_known()	"up"	SafetyResult(passed=True)
—	check_within_envelope()	z=264 vs z_max=350	SafetyResult(passed=True)
4 — EXEC	move_to_position()	proposed (z=264)	HTTP GET → arm moves → 200 OK
5 — LOG	mark_s1() + log()	proposed	proposed.state_type = S1
—	reset_to_s0()	proposed (S1)	proposed.state_type = S0, new S0

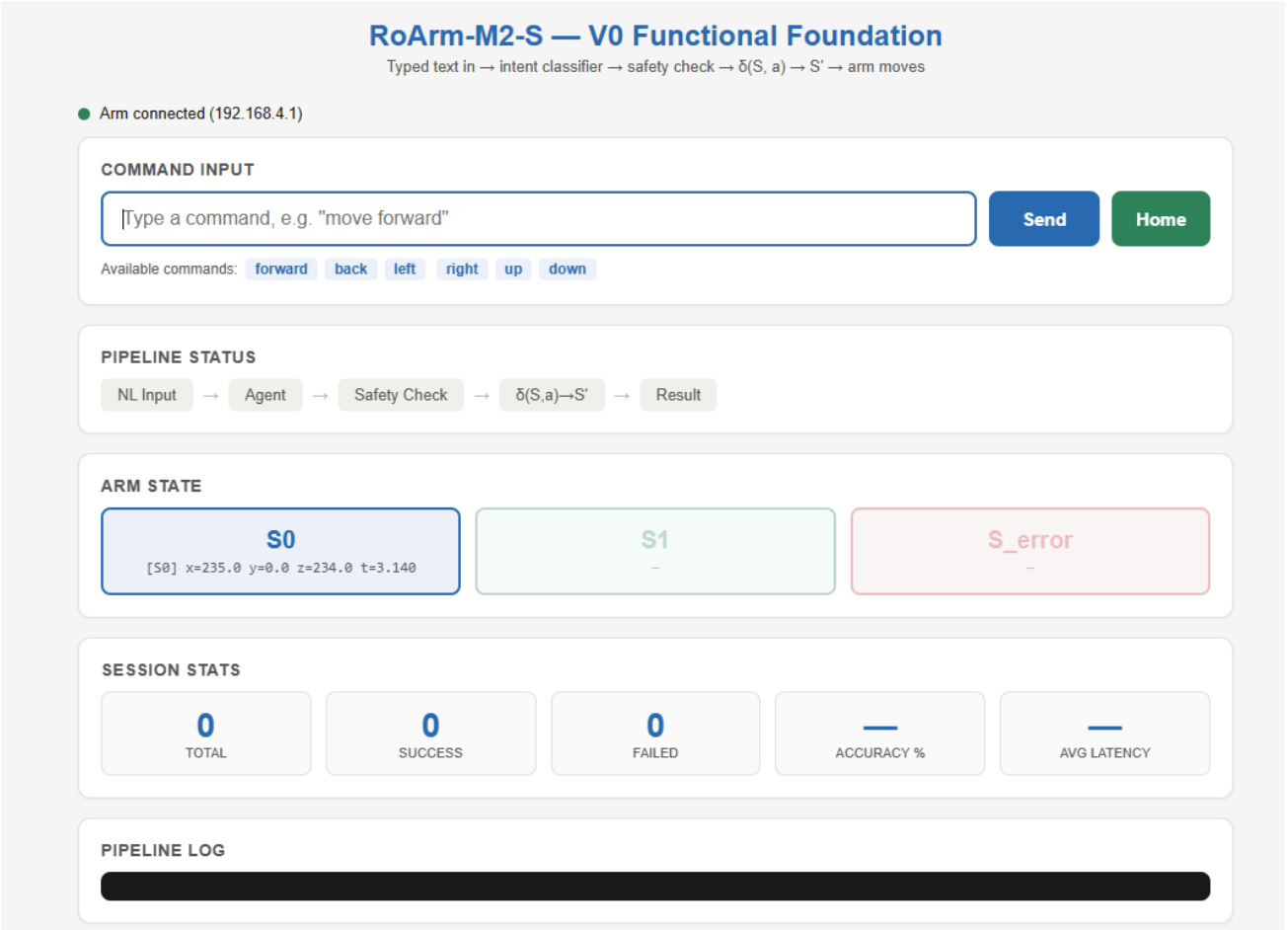
HOW IT WORKS

The phrase "lift it up a bit" matches the \blift\b pattern in the "up" group. The 'a bit' part is simply ignored — the classifier only looks for the direction keyword. Step size is always STEP_SIZE_MM regardless of qualifiers like 'a bit' or 'a lot'. Adding magnitude parsing is a V1 task.

11. V0 Success Criteria

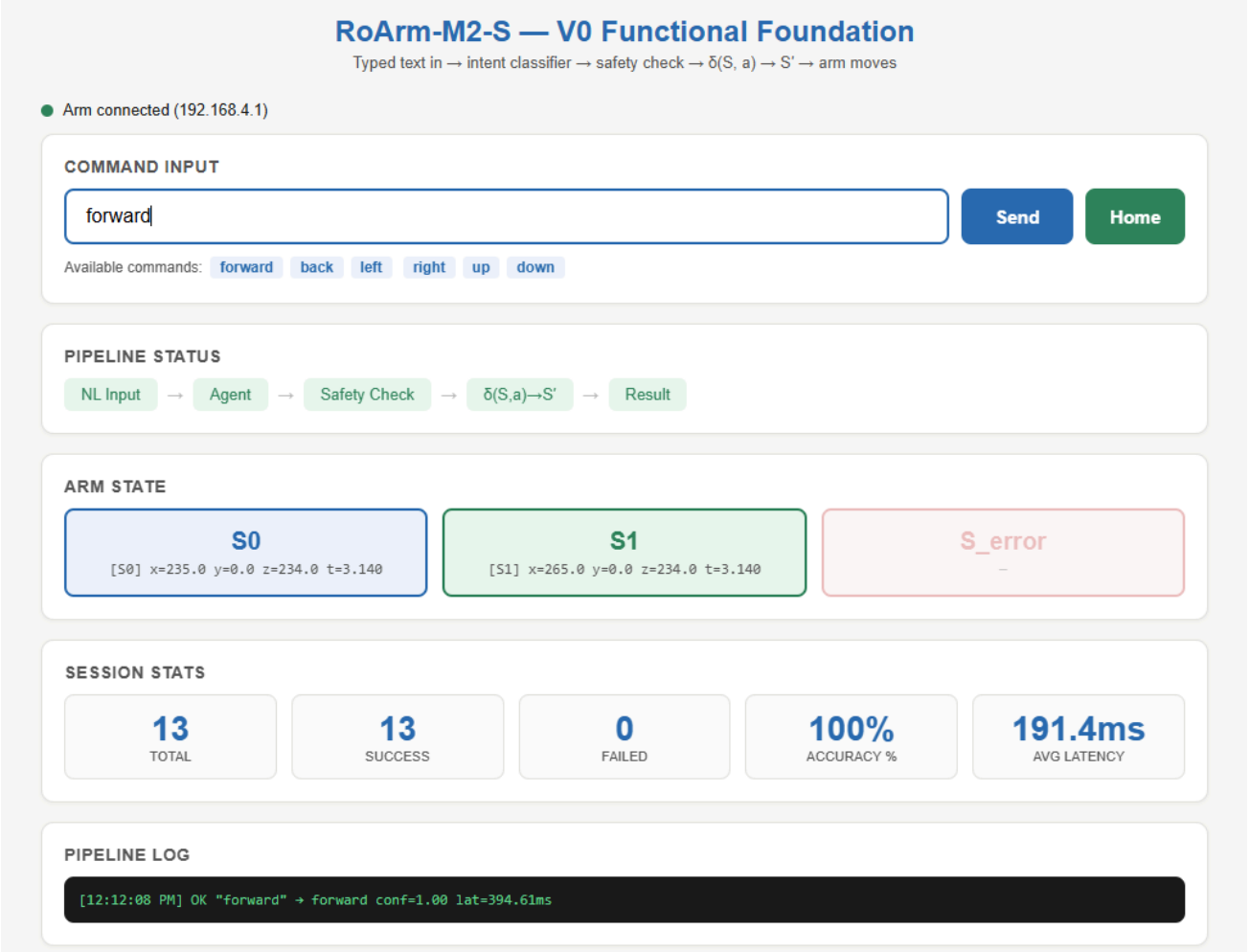
V0 was declared done when all of the following were verifiable on hardware:

Criterion	How to verify	Status
1. 6All 6 directions classified correctly	Run each direction word through classify_intent()	✓ Pass
2. Unknown input → S_error, no arm movement	Type 'dance around', check arm stays still	✓ Pass
3. Out-of-bounds → safety fail, no movement	Set proposed z > z_max, check execution skipped	✓ Pass
4. WiFi disconnect → exec fail gracefully	Unplug arm, type 'up', check error returned	✓ Pass
5. S_error → auto-reset to S0	Check state_type after any failure	✓ Pass
6. Latency < 500ms per command	Avg latency shown in /stats, expect ~260ms	✓ Pass
7. Stats endpoint returns accuracy %	GET /stats after 8 commands	✓ Pass
8. Browser UI shows live pipeline status	Visual step indicators light up green/red	✓ Pass

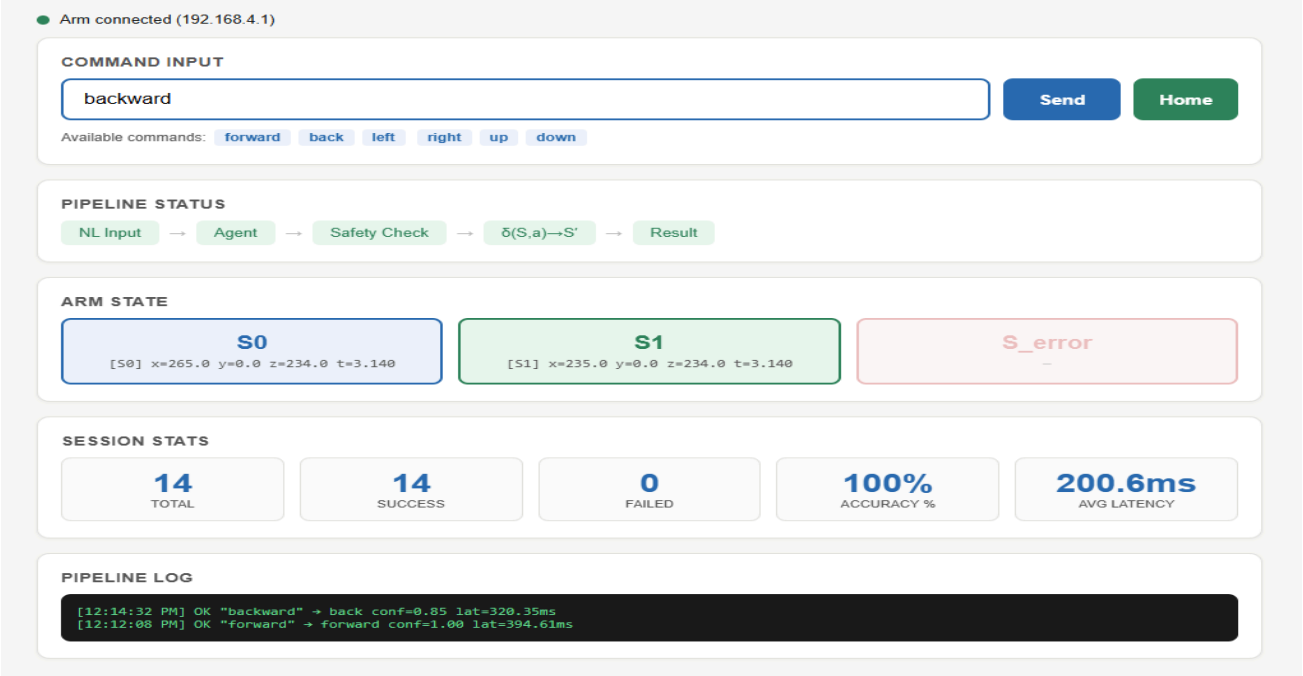


Initial Home Screen

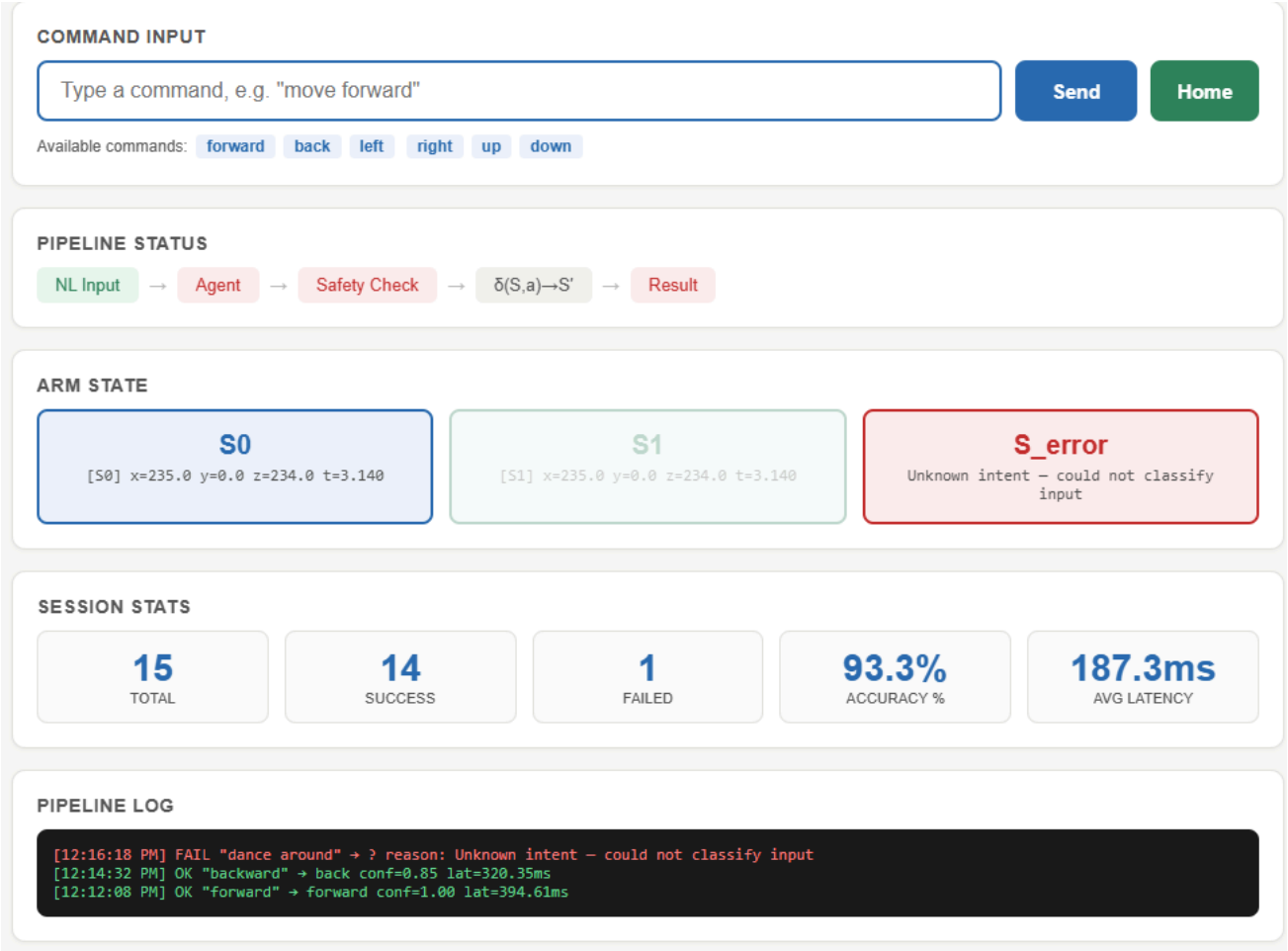
Success Criteria Verification



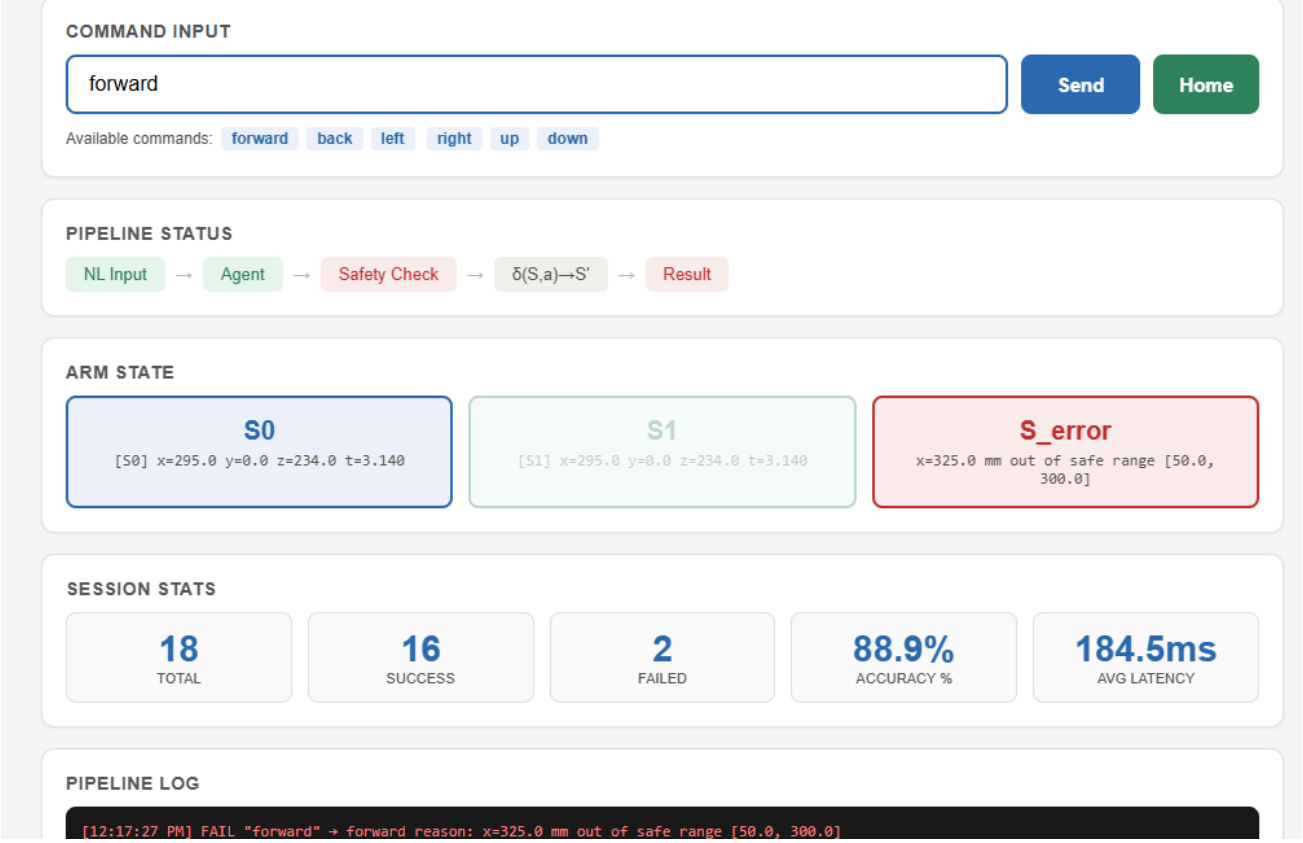
1-Forward Command



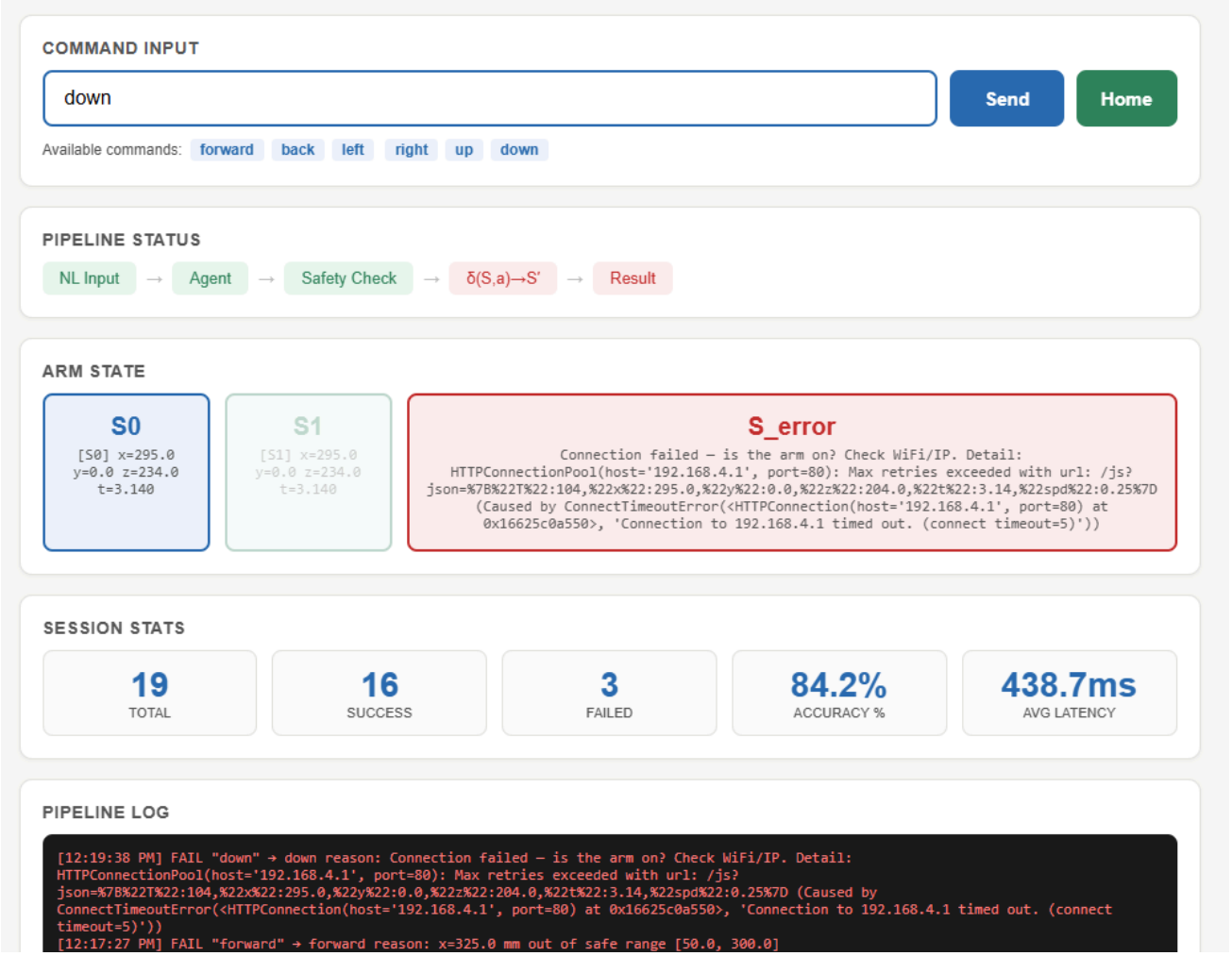
1-Backward Command



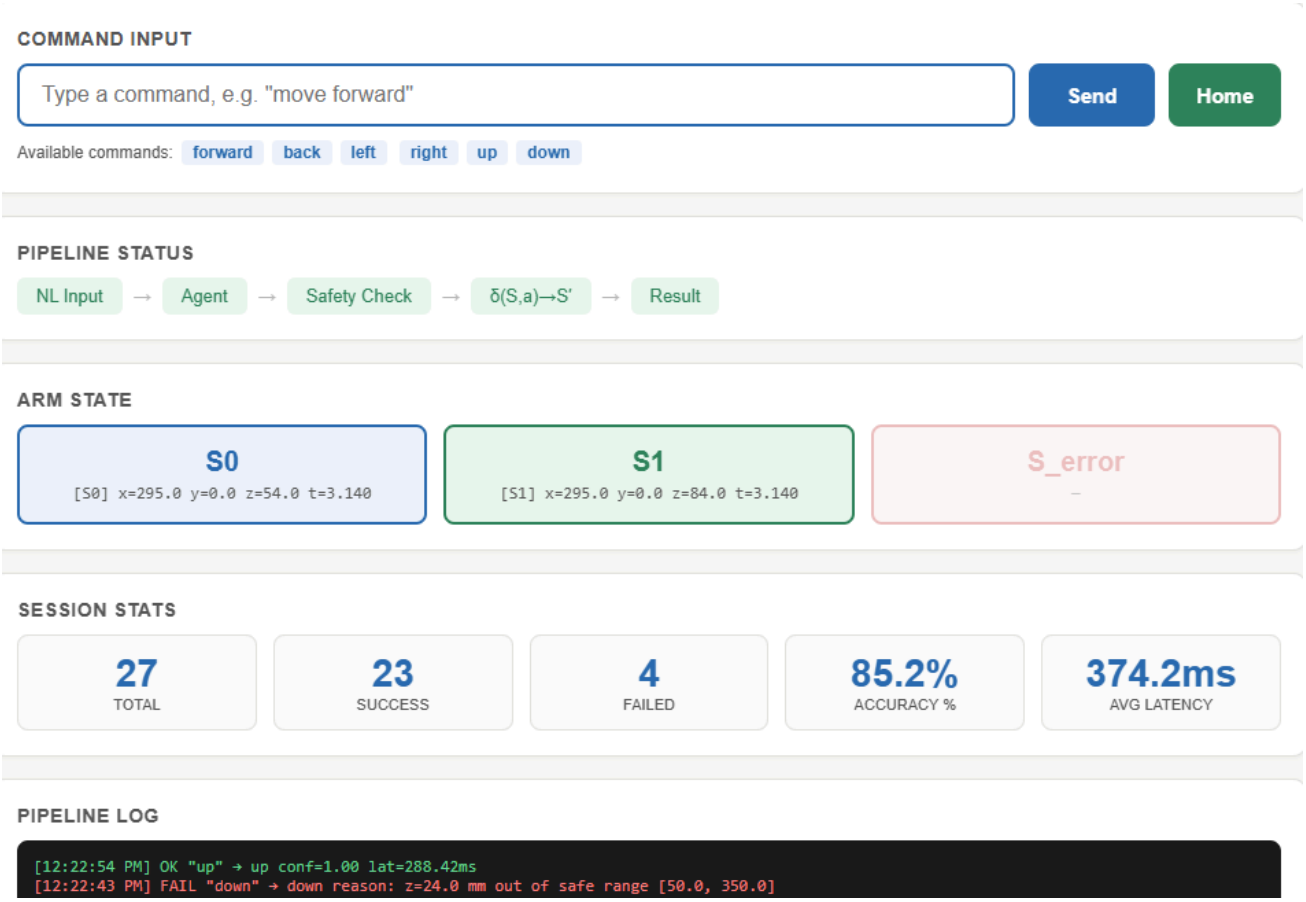
2-Dance Around Fails



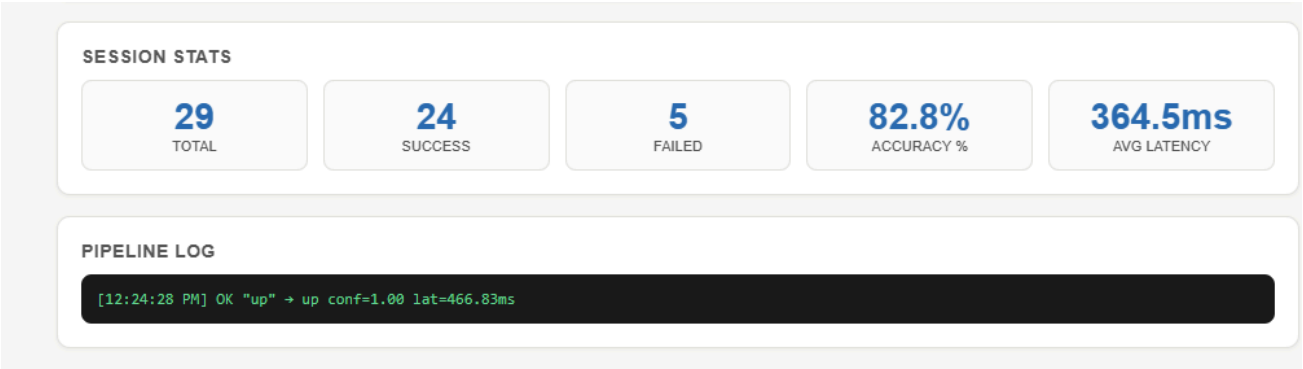
3-Out-of-bound



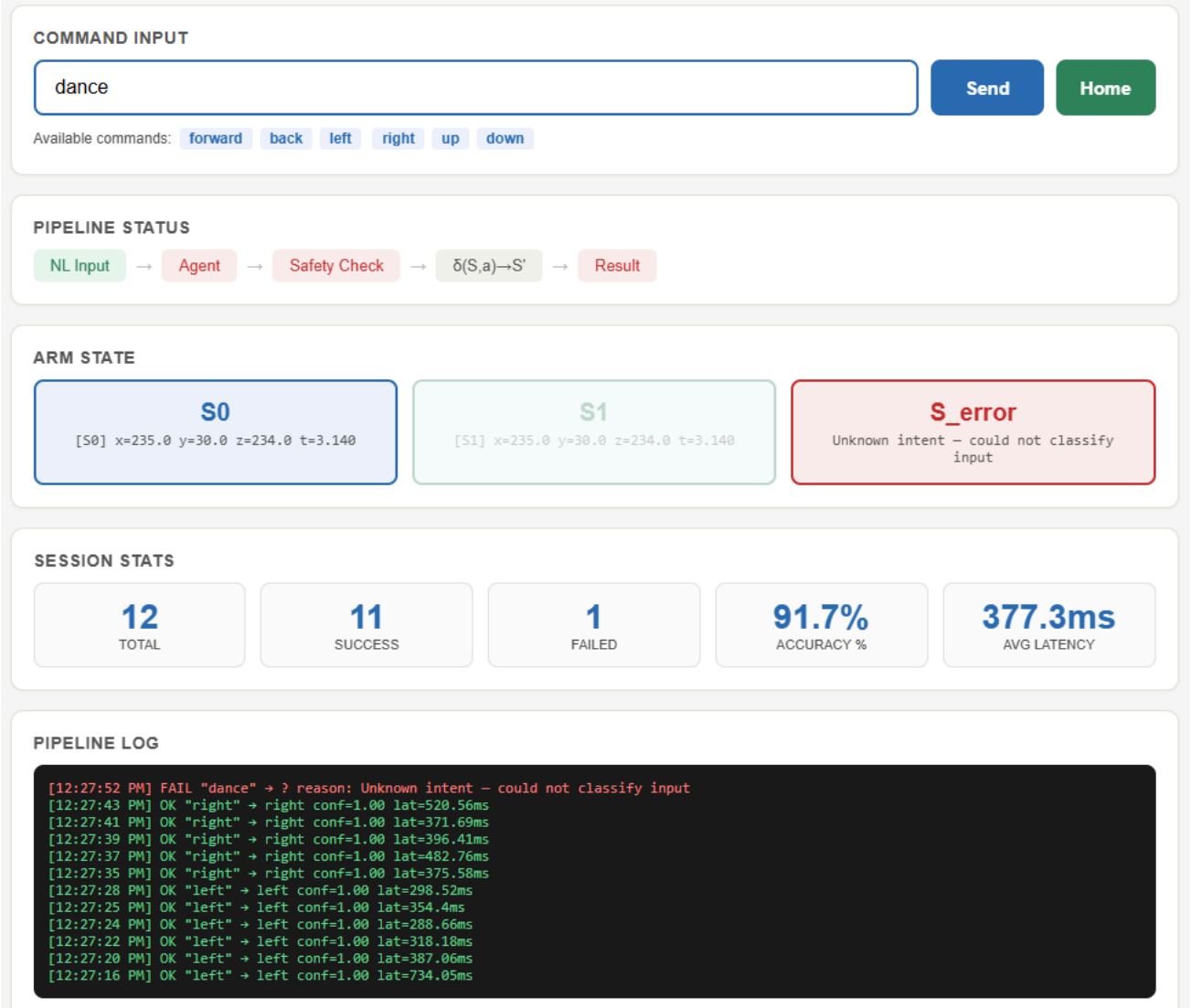
4-WiFi disconnect ⟶ exec fail gracefully



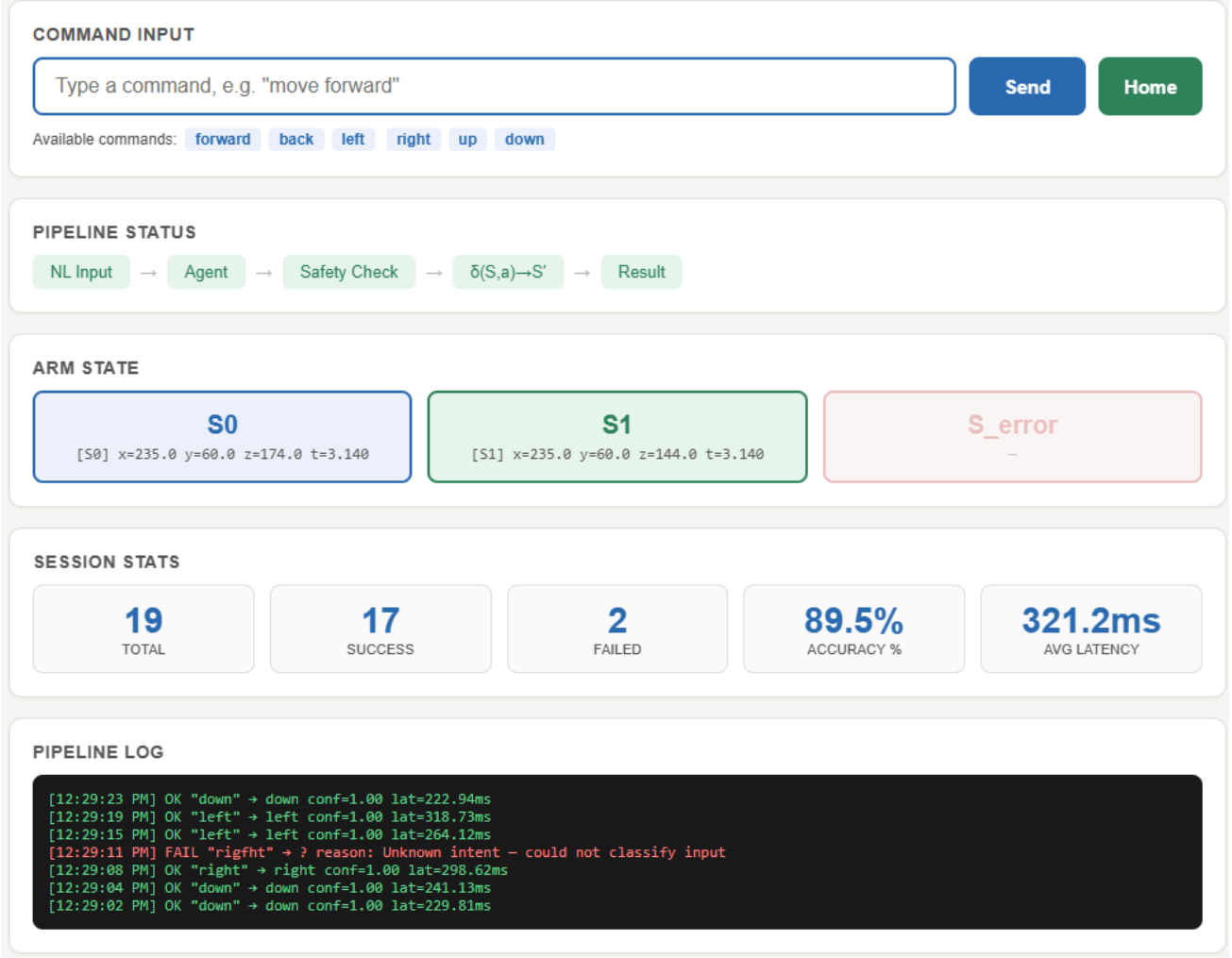
5. S_error ⟶ auto-reset to S0



6. Latency < 500ms per command



7. Stats endpoint returns accuracy %



8. Browser UI shows live pipeline status